

🎯 GUIA: 3 Alertas Separados — Sniper + RSI + Catalisador (5m)

📋 Como o bot decide (o fluxo que você descreveu)

O bot **já foi construído** para receber **3 alertas SEPARADOS** (nada de combinar indicadores):

1) SNIPER (rosa x azul) → /webhook/<MOEDA>
↳ O bot GUARDA o sinal como "pendente" (segura, aguardando o RSI)

2) RSI cruzou a média → /rsi/<MOEDA>
↳ Se há sinal do Sniper pendente E o RSI cruzou na direção certa (dentro da janela de 3-4 velas) → o bot decide ENTRAR

3) CATALISADOR (contexto multi-TF) → /catalyst
↳ ANTES de simular a entrada, checka se o contexto está a favor. Se NÃO estiver → BLOQUEIA. Se estiver → simula a entrada.

Resumindo: Sniper segura → RSI confirma → Catalisador libera ou bloqueia. Exatamente o que você pediu, e cada um é um alerta próprio.

Isso está confirmado no código:

- engine.on_signal() → cria o **sinal pendente** (passo 1)
- engine.on_rsi_cross() → confirma o pendente e decide **entrar** (passo 2)
- crypto_shadow._checar_gates() → roda catalyst.checar() e **bloqueia se contra** (passo 3)

🔔 ALERTA 1 — Sniper TSTS (rosa x azul)

Este é o **seu indicador** (o TSTS Sniper que já usa o cruzamento da linha rosa x azul). Você **não precisa mudar o indicador** — só criar o alerta apontando para o bot.

Configuração

Campo	Valor
Condição	Seu indicador Sniper → o sinal de cruzamento (buy/sell)
Opções	☒ Once Per Bar Close
Webhook	https://web-produc-tion-77454.up.railway.app/webhook/{{ticker}}
Nome	[5m] Sniper {{ticker}}

Mensagem (JSON)

Se o seu indicador Sniper **não monta JSON sozinho**, cole isto no campo Message do alerta:

Para o alerta de COMPRA:

```
{"moeda":"{{ticker}}","action":"buy","timeframe":"{{interval}}"}
```

Para o alerta de VENDA:

```
{"moeda":"{{ticker}}","action":"sell","timeframe":"{{interval}}"}
```

O bot aceita `action` : `buy` / `sell` (também `long` / `short` , `compra` / `venda` , `1` / `-1`).



ALERTA 2 — RSI cruzando a média

Use o Pine `rsi_cross_sniper.pine` (que criei) — é um indicador **simples e independente** que dispara quando o RSI cruza a média.

⚠ Use o **mesmo período de RSI e a mesma média** do seu setup do Sniper, para a confirmação ser coerente.

Configuração

Campo	Valor
Indicador	RSI Cross – Sniper (cole o Pine no gráfico)
Condição	RSI Cross – Sniper → Any alert() function call
Opções	☒ Once Per Bar Close
Webhook	<code>https://web-production-77454.up.railway.app/rsi/{{ticker}}</code>
Mensagem	deixe VAZIO (o Pine já monta o JSON)
Nome	[5m] RSI {{ticker}}

JSON que o Pine envia (automático)

```
{"moeda":"BTCUSD", "direction":"up", "timeframe":"5m", "rsi":58.3, "rsi_ma":55.1}
```

- `direction: up` → confirma **compra**
- `direction: down` → confirma **venda**

ALERTA 3 — Catalisador (contexto multi-TF)

Use o Pine `mnq_catalyst_v2_cripto.pine` — ele lê os timeframes (30s/1m/5m/15m/1h + **2h/4h**), VWAP, regime de mercado e pullback, e manda o contexto por moeda.

Configuração

Campo	Valor
Indicador	MNQ Catalyst V2 – CRIPTO (SNIPER) (cole o Pine no gráfico)
Condição	esse indicador → Any alert() function call
Opções	☒ Once Per Bar Close
Webhook	<code>https://web-production-77454.up.railway.app/catalyst</code>
Mensagem	deixe VAZIO (o Pine manda o JSON com <code>moeda</code> dentro)
Nome	[5m] Catalyst <code>{{ticker}}</code>

O `/catalyst` (sem moeda na URL) funciona porque o JSON já traz `"moeda": "{{ticker}}"`. Um alerta serve para **qualquer** moeda.

JSON que o Pine envia (automático)

```
{
  "moeda": "BTCUSDT",
  "c30s": "BULL",
  "c1m": "BULL",
  "c5m": "BULL",
  "c15m": "BULL",
  "c1h": "NEUT",
  "c2h": "BULL",
  "c4h": "BULL",
  "vwap": "BULL",
  "market": "TRENDING",
  "pullback": "NONE",
  "grade": "B"
}
```

Importante: o catalisador **não gera entrada sozinho** — ele só atualiza o contexto da moeda. A entrada só é avaliada quando o Sniper + RSI disnapam.

Ordem de configuração recomendada

1. **Catalisador primeiro** (Alerta 3) — assim o contexto já está “quente” quando o sinal chegar.
2. **RSI** (Alerta 2).
3. **Sniper** (Alerta 1) — o gatilho principal.

Faça isso **por moeda** (o mesmo gráfico serve para os 3 alertas da moeda).

✓ Testar o fluxo completo

1. Confirmar recepção

```
curl https://web-production-77454.up.railway.app/diag
```

Procure em `ultimos_eventos` por `webhook_sinal`, `webhook_rsi` e `catalyst`.

2. Ver o contexto do catalisador de uma moeda

```
curl https://web-production-77454.up.railway.app/diag
```

(o bloco do catalisador aparece por moeda no diagnóstico)

3. Ver as entradas simuladas

```
curl https://web-production-77454.up.railway.app/registro
```

Deve mostrar `ENTRADA` (liberada) ou `BLOQUEADO` (catalisador barrou), com `regra` e `grade`.

4. Estudo de TP/SL (Postgres — durável)

```
curl https://web-production-77454.up.railway.app/estudo
```

`diag_backend.driver` deve estar `pg8000` e `use_pg: true` (persistência ativa).

⚙️ Ajustes finos (config.json → bloco `catalyst`)

Chave	Padrão	O que faz
<code>ativa</code>	<code>true</code>	Liga/desliga o catalisador como gate
<code>bloquear_ranging</code>	<code>false</code>	Removido a seu pedido (não bloqueia lateralização)
<code>pullback_ativo</code>	<code>true</code>	Só entra na retomada após pullback
<code>timing_rapido</code>	<code>true</code>	Usa 1m/5m como confirmação de micro-timing
<code>bloquear_contra_macro</code>	<code>false</code>	Deixe false por ora (coletando dados 2h/4h). Ligue quando for pro real

Troubleshooting

Bot responde `ignorado` / `MODO AUTÔNOMO`

- O bot está com `aceitar_webhooks=false`. Para receber os alertas do TradingView, precisa estar `true` no config. (Me avise que eu ajusto.)

Alerta do Sniper chega mas nunca entra

- Verifique se o RSI cruzou **na direção certa** dentro da janela (o `on_rsi_cross` confirma o pendente).
- Veja em `/registro` se saiu `AGUARDAR` (esperando RSI) ou `BLOQUEADO` (catalisador barrou).

Sempre `BLOQUEADO` pelo catalisador

- Confira `bloquear_ranging=false` e `bloquear_contra_macro=false`.
- Veja o `motivo` e a `regra` no evento `BLOQUEADO` do `/registro`.

RSI não dispara

- Confirme o mesmo período/média do RSI do seu setup.
 - Teste com o `maTipo` = SMA (padrão do seu setup).
-

Checklist de ativação (por moeda)

- [] Alerta 3 (Catalisador) criado → `/catalyst`, mensagem vazia
 - [] Alerta 2 (RSI) criado → `/rsi/{{ticker}}`, mensagem vazia
 - [] Alerta 1 (Sniper) criado → `/webhook/{{ticker}}`, JSON com `action`
 - [] Todos com “Once Per Bar Close”
 - [] `/diag` mostra os 3 eventos chegando
 - [] `/registro` mostra `ENTRADA` ou `BLOQUEADO` com regra/grade
 - [] `/estudo` mostra `driver: pg8000`, `use_pg: true`
-

Depois de rodar alguns dias

1. Trago o **win rate por regra e por grade** (A/B/C).
2. Decidimos quais regras podar (R3 fallback, grade C, R9 são candidatos).
3. Calibramos TP/SL com **MFE/DD real** (dados do Postgres).
4. Quando for pro real em 5m: ligar `bloquear_contra_macro: true`, começar com **5x** (10x só grade A).